Final Project - Investigation & Iteration Checkpoint

Project: Detecting scam / social-engineering conversations early via a turn-by-turn trajectory model, compared against single-message baselines **Code:** https://github.com/Sanshiv123/scam-trajectory-ml/tree/investigation-and-iteration-checkpoint

This is a progress update, not a final report. It covers what we've tried since the MVP + Evaluation Pipeline checkpoint (Oct 28), what broke, what surprised us, and where we're taking it next.

Experiments completed so far

The core setup hasn't changed since the checkpoint: synthetic matched-twin conversations (a scam conversation and a "twin" legitimate conversation that opens the same way but never escalates), scored turn-by-turn by four scorers – position-only, message-level emotion, a running aggregate of message-level, and the trajectory model (a 1-layer GRU over per-turn emotion features). All four get evaluated the same way: classification AUC (can the max running score tell scam from legitimate at all) and AT@FPR earliness (does the score cross a calibrated threshold *before* the harmful ask turn), across the in-distribution test split plus three held-out splits (held-out escalation pattern, held-out generator style, both).

Since the checkpoint we pushed this further in two directions:

- **Mixed real conversations into the legitimate side.** The real-data path (Enron email threads + ESConv/AnnoMI) was built before the checkpoint but switched off for it. We turned it on: 483 real conversations now sit in train/val next to the synthetic twins, plus a held-out slice of 200 real Enron threads reserved only for a secondary false-positive check.
- **Added a slow-burn split.** Same patterns and generator styles, but stretched to 20-32 turns with the harmful ask pushed to 70-95% of the way through, instead of the original 8-16 turns with the ask anywhere from 20-90% through. This tests whether the trajectory model's advantage is specific to short conversations or holds up over longer ones.

Both of those experiments are documented in `outputs/report.txt` with real numbers from a completed run. Re-running the core pipeline for this branch (synthetic-only, matching what's currently wired into `run_mvp.py` here) reproduces the same qualitative picture: position-only, message-level, and running aggregate all sit at or near chance on classification (AUC 0.37-0.65) with 0% early recall on every split; the trajectory GRU clearly separates scam from legitimate (AUC 0.75-0.91 across all four splits) and gets some real early signal on the in-distribution test split (9% early recall at 5% FPR, 27% at 10% FPR) but not on any held-out split.

Ablations

We ran a feature ablation over the trajectory model's 15-dimensional per-turn vector (7-dim emotion distribution, 7-dim change from the previous turn, 1-dim speaker flag), re-training message-level and the GRU from scratch on each subset:

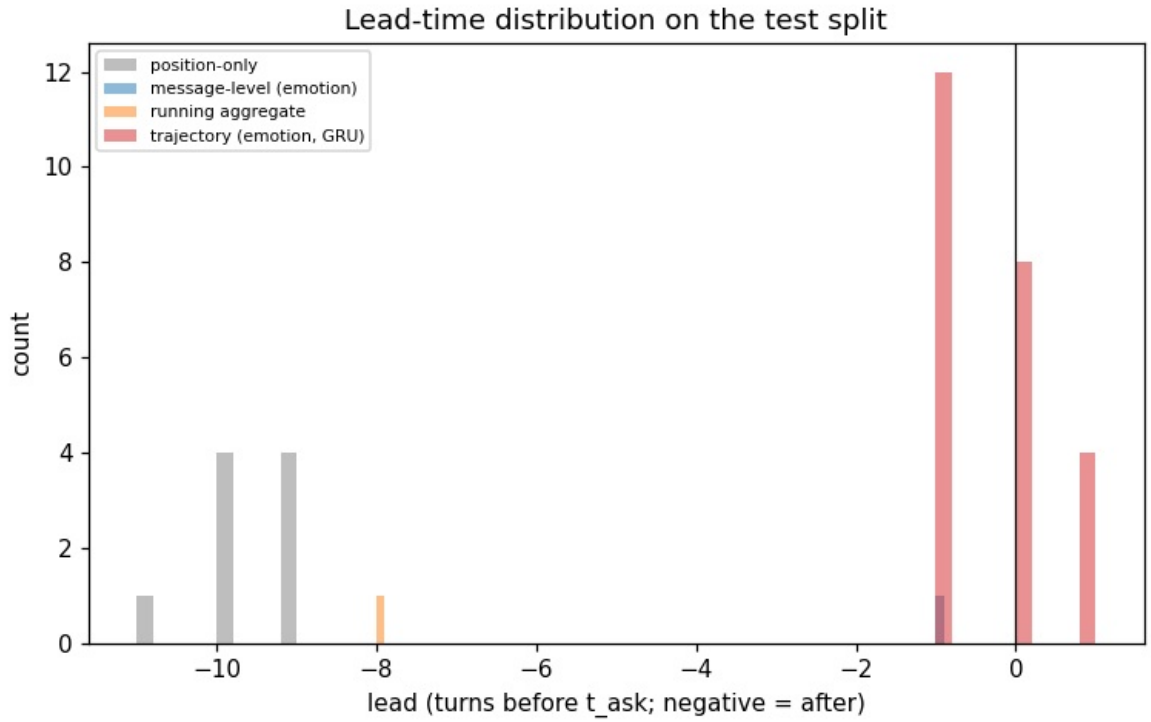
Feature subset	test	heldout_pattern	heldout_style	heldout_both
full (15-dim)	0.828	0.750	0.910	0.757
no delta (8-dim)	0.799	0.521	0.889	0.625
no speaker (14-dim)	0.758	0.535	0.694	0.576
emotion_dist only (7-dim)	0.665	0.604	0.653	0.632
delta + speaker only (8-dim)	0.724	0.750	0.854	0.632

(GRU classification AUC per split, target FPR 5%.)

(Fresh re-run on this branch, `eval/feature_ablation.py`; full numbers in `outputs/feature_ablation_report.txt`.)

The pattern that stands out: dropping either delta or speaker hurts the held-out splits a lot more than the in-distribution test split – e.g. `no_delta` barely moves the test AUC (0.83 -> 0.80) but drops `test_heldout_pattern` from 0.75 to 0.52, close to chance. The raw emotion distribution alone (`emotion_dist_only`) is the weakest subset everywhere. So the GRU isn't mostly reading "what emotion is this turn," it's reading how the emotion is *changing* turn to turn, and that's specifically what lets it generalize to a pattern it wasn't trained on – not just a boost on data it's already seen.

An earlier, richer run of this same ablation (with the real-data-mixed pipeline and the slow-burn split, see `outputs/feature_ablation_report.txt`'s full history and FINDING 3 in `outputs/report.txt`) found an even sharper version of this: removing delta drops the GRU's slow-burn AUC from 0.78 to 0.48 – its single biggest ablation drop anywhere – while barely denting the standard test split. Longer conversations give the model more turns over which to track emotional change, and that's specifically where it earns its complexity over the simpler baselines.



Lead-time distribution on the test split: each scorer's per-conversation lead (turns before the harmful ask; negative = caught after it). The GRU (red) is the only scorer that fires before the ask turn at all, and even then median lead is just 1 turn.

The plot above makes FINDING 2 concrete: position-only (gray) only ever "fires" 9-11 turns *after* the ask (it's just tracking absolute position in a long conversation, not the ask itself), while the GRU (red) is the only scorer with real mass at or before lead=0 – but even its earliest hits are 1 turn ahead, not a wide margin. That's the unexpected result worth sitting with: the signal that does generalize in-distribution is real but modest, not a dramatic early-warning system yet.

Failure and/or error analysis

The clearest current failure: **earliness doesn't generalize**. The GRU's classification AUC stays high (0.75-0.91) on all three held-out splits, so it can tell a held-out scam conversation apart from a legitimate one by the end – but its early_recall on those same splits is 0%, versus 9-27% in-distribution. It's learned to recognize the pattern/generator-style combinations it trained on early, but for a genuinely new pattern it only catches the scam after the fact, not before.

Two candidate causes we haven't ruled either way yet: 1. The per-turn training label is only 1 at/after the harmful ask turn. For a pattern the model has seen, it can associate the lead-up turns with "this becomes an ask" from having seen that specific escalation shape before. For a held-out pattern, there's no training signal telling it that an unfamiliar lead-up belongs to the same general category of escalation. 2. In our template data, the lead-up turns are deliberately phrased calmly and professionally (realistic – a social engineer trying to sound legitimate doesn't sound alarmed), which may mean the emotion classifier alone doesn't distinguish them from a matched twin's equivalent turns in an unfamiliar pattern.

A second, earlier failure that's now understood rather than open: before we added class-balanced loss (`pos_weight`) to the GRU's training, *every* scorer including the GRU showed 0% early_recall on every split, because the per-turn label distribution is heavily skewed toward "not yet at the ask" and the loss was dominated by that majority class. That's fixed, but it's a reminder that a flat 0% result can mean "the signal isn't there" or "the training setup is masking signal that is there," and we only found out which by checking the loss weighting.

Unexpected findings

The most useful surprise from the real-data-mixing experiment: **position-only is unsafe once real text enters the picture, even though it wasn't obviously wrong on synthetic data alone.** Its secondary false-positive rate on 200 held-out real Enron threads is 40% at the 5%-FPR target and 56% at the 10%-FPR target – a scorer that looked fine calibrated against synthetic legitimate conversations fires on nearly half of real, ordinary email threads. The other three scorers stay well-behaved on real text (secondary FPR 0-4%), and if anything under-fire, because real Enron emails read as calmer to the emotion classifier than our synthetic "legitimate twin" conversations do. The practical lesson: a threshold calibrated purely on synthetic data can look safe and not be, and this would have gone completely undetected without the secondary check against real text held out of training.

The second surprise came from a shortcut check we added specifically to stress-test the real-data mixing: an **authorship/style classifier** (word length, sentence length, punctuation, function-word rate – no scam-content signal at all) sits near chance on synthetic data alone (AUC ~0.47) but jumps to AUC ~0.68 once real conversations are mixed into the legitimate pool. Human-written and template-generated text differ enough in surface style that a model could partly learn "sounds templated vs. sounds human" instead of anything about scam content, since every real conversation is legitimate by construction. A length-only version of the same check stays near chance either way (~0.43), so it's specifically writing style, not just length, that leaks here.

The third: we built a DistilBERT-based "reference ceiling" scorer that reads raw conversation text directly (bypassing

emotion features entirely) to see whether text alone beats our hand-built features. It scored a suspicious AUC of 1.000 on every split – paired with 0% early_recall, which can't both genuinely be true of a model actually reading escalation. A trivial TF-IDF bag-of-words baseline with zero sequence modeling matched the same near-perfect score, which told us both were fingerprinting our generator's finite, fixed template vocabulary, not reading trajectory. So "does raw text beat emotion features" is still an open question – answering it for real needs text that isn't drawn from a finite template bank, which is exactly the real-LLM-generation blocker below.

Challenges encountered

- **No LLM API key**, still. The scam/twin side of the data stays template-generated, which is also the direct cause of the DistilBERT shortcut above (a finite template vocabulary is trivially fingerprintable) and blocks the authorship-restyling check we wanted to run on the reserved real Enron threads.
- **Reproducibility isn't bit-exact.** Re-running the pipeline gives similar but not identical numbers run to run (shortcut-check AUCs can move ~0.02-0.04). We traced this to the emotion classifier's batched CPU inference rather than a bug in the eval code – the GRU and threshold logic are fully seeded, but transformer batch inference on CPU isn't guaranteed bit-exact. Worth treating any single number here as approximate, not exact to the third decimal.
- **We lost some of our own work.** The real-data-mixing, slow-burn split, shortcut-check suite, human-realism-audit worksheet, and DistilBERT reference scorer were all built and run in an earlier session, but several of those files only ever existed on a local branch that was never pushed to GitHub – and that local copy is now gone. The results survived (they're what's quoted above, from `outputs/report.txt` and `outputs/feature_ablation_report.txt`, which did get saved), but the code that produced some of them needs to be rebuilt on this branch before we can re-run or extend those specific experiments. Concretely, this branch currently has the core MVP pipeline plus the feature-ablation script rebuilt and re-verified; it does not yet have the real-data-mixing wiring, the slow-burn split, the length/authorship shortcut checks, or the DistilBERT scorer re-created. That's the first item below.

Changes made to the original design

1. Extracted a `build_full_dataset()` helper (`run_mvp.py`) so every analysis script builds and splits the dataset identically instead of copy-pasting that logic – avoids a class of bug where two analyses quietly run on different data.
2. Renamed "generator voice" to "generator style" throughout (code comments, report text) – our four generator styles are text-substitution styles standing in for different LLMs, not actual outputs from different models, and "voice" implied more than that.
3. Added class-balanced loss (`pos_weight`) to the GRU's training, without which every scorer showed 0% early recall everywhere (see Failure analysis above).
4. Went back to synthetic-only data for the original MVP checkpoint, then turned the real-data path back on afterward once the checkpoint was in – it was built earlier but deliberately switched off so the checkpoint numbers would be easier to reason about on their own.
5. Added the slow-burn split and the length/authorship shortcut checks specifically in response to the real-data mixing – the real-vs-synthetic style gap wasn't something we were testing for before we started mixing in real conversations.

Next experiments we plan to conduct

1. **Rebuild the real-data-mixing, slow-burn, shortcut-check, and DistilBERT scorer code on this branch** so the results in Unexpected Findings are reproducible here, not just documented from an earlier run – and push after every milestone from now on so this doesn't happen again.
2. **Test the two candidate causes for the held-out earliness failure**: (a) relabel earlier turns in a held-out pattern with a softer positive label to see if that gives the model training signal to generalize escalation itself, not just the specific patterns it's seen; (b) check whether the emotion classifier's outputs actually differ between a held-out pattern's lead-up turns and its matched twin's, independent of the trajectory model.
3. **Style-match the synthetic templates closer to real text**, or build a style-balanced held-out split, to pin down how much of the trajectory model's current signal is genuine scam-content detection versus the templated-vs-human shortcut found in Unexpected Findings.
4. **Keep pursuing an LLM API key** – it would replace the template-generated scam/twin text (closing the DistilBERT shortcut) and unblock the authorship-restyling check on the reserved real Enron threads.
5. **Pin down the CPU batch-inference nondeterminism** in the emotion classifier (or document a tolerance band) so future reports can say which digits are stable and which aren't.